2AMM10 Assignment 1: Learning Garden Features

Friday 24th April, 2026

Introduction

Hannah Garden is a passionate gardening enthusiast who has cultivated an impressive collection of fruits and vegetables. Inspired by recent advances in AI, she wants to build a smart system capable of recognizing and organizing the produce in her garden. Rather than relying on rigid, hand-crafted categories, Hannah envisions a model that learns meaningful visual representations of her plants.

As AI experts, you have been recruited to develop this system. Your challenge is to design models that can extract and learn useful features from images of garden produce, making it possible to not only categorize known items but also recognize unseen ones. The project consists of multiple tasks, gradually increasing in complexity to build a robust solution.

The Dataset

Hannah is too busy at the moment to collect pictures from her garden so for this proof of concept, you will be relying on the *Fruits-360 dataset*. She adapted the dataset to have three hierarchy levels:

- **Family:** The coarsest level of categorization (e.g., Apple, Tomato, Cucumber, ...)
- **Item:** Within each family, several distinct items are present (e.g., different varieties of apples)
- **Frame:** Each item is rotated in front of a camera, and frames are captured at different angles

The test set consists of images with camera angles not seen during training[1]. For a visualization of the samples in the dataset and the underlying hierarchy, you can use the visualization tool in the Jupyter notebook (section"Task 2").

Task 1

As a warm-up, we have created a subset of the fruits dataset called the *AppleDataset*. This dataset is composed of 30 different apple items, 20 of which are used for training. The goal is to design a neural network that learns embeddings of apple images, such that pictures of the same item yield similar embeddings while pictures of different items yield different embeddings. You are asked to do the following:

1. Visualize the dataset images using the tool available in the provided Jupyter Notebook and think of an appropriate neural network architecture to learn embeddings from this dataset.
2. Describe an appropriate loss function that satisfies the goal of learning similar embeddings for images from the same item and different embeddings for images from different items.

[1] See `https://github.com/fruits-360/fruits-360-original-size` for more details

3. Implement your model and an appropriate training procedure.

4. Given an image of one of the apples captured at a camera angle not seen during training, how can you use your model and the training set to predict which item the image corresponds to? Implement your method and evaluate its accuracy.

5. How can you handle items not seen during training? You are given a dataset containing images from the 10 apples not included in training, along with their corresponding labels (support set). How can you use this support set and your model to classify images from one of these 10 apples, without retraining the model? Implement your method and evaluate its accuracy.

You are expected to reach an accuracy above 90% for the seen items and above 80% for the unseen items.

Task 2

You are now ready to train an embedding model on the full dataset. The *GardenDataset* training set includes 15 families and 69 items. Using the same loss as in Task 1, train a model[2] to obtain close embeddings for images of the same item. Report the classification performance of your model in the scenarios described below:

Scenario	Support set	Test set	Classification level
1	Train	Main classes test	Items
2	Train	Main classes test	Families
3	Train + new items	New items test	Items
4	Train + new families	New families test	Families

The support set is used to support classification via provided labels. It always includes the training set and may additionally include items or families not seen during training. Despite the model being trained on items, we also want to evaluate its ability to perform coarse family-level classification. For each scenario, compute the accuracy and, in order to better assess the more challenging cases, do the following:

1. In scenario 1, visualize[3] how the item classification accuracy varies across families. Which families are the most challenging?

2. In scenario 2, visualize the confusion matrix to assess which families are most likely to be confused by your model.

You are expected to reach an accuracy of at least 90% in the scenarios 1, 2 and 3, and 80% in scenario 4.

Task 3

Until now, we have only trained models using item-level labels. However, Hannah Garden would be disappointed if the model failed to correctly predict the family of an input. To improve performance at the family level, we need an **additional** loss term that encourages embeddings from the same family to be closer to each other than embeddings from different families. How would you augment the previous loss to include this new objective? Note that the original item-level objective must be preserved.

After training your model with this combined loss, evaluate its performance using the same scenarios as in Task 2. Since the additional loss is specifically designed to improve family-level classification, your model should achieve better or equal accuracy in scenarios 2 and 4 compared to Task 2, while not significantly degrading performance in scenarios 1 and 3.

[2] You may adapt the architecture for the full dataset.
[3] using bar plot, for example.

Task 4

Building a large annotated dataset is time-consuming, and Hannah Garden is considering collaborating with active users of her gardening blog to create a "community garden dataset". However, she is concerned about the quality of images that contributors might submit. She has asked you to assess whether it is possible to build a system capable of detecting abnormal samples. To emulate corrupted images, Hannah has created datasets derived from the *GardenDataset* test set in which a fraction of each image's pixels are set to black at random locations.

Using the training set and the model obtained in Task 3, how could you build an anomaly detection method? Implement such a method. If we tolerate 10% of normal test samples being wrongly flagged as anomalies, what percentage of anomalies would be detected for the following black pixels fractions: 1%, 5%, and 10%? Given the limitations of using an embedding model for this task, the expected detection rates are relatively modest: all fractions should yield a detection rate above 10%, and at least 40% and 50% for 5% and 10% of black pixels respectively.

Using a feature embedding model for anomaly detection may not be the most robust solution, as the embedding model was trained with a different objective. To decouple feature embedding from anomaly detection, one alternative is to train a model dedicated solely to this task. Since anomalies are rare in practice and their nature is not always known in advance, such models are typically trained exclusively on normal data. How could you design an Autoencoder trained on the *GardenDataset* training set to serve this purpose? Implement such a method and evaluate its performance in the same setup as before. The autoencoder should significantly outperform the embedding-based approach: you are expected to detect more than 75% of anomalies with 1% of black pixels, and more than 90% with 5% of black pixels.

Deliverables

The submission of this assignment is done in ANS. There are two deliverables:

- Implementation of the solution
- Description and explanation of your approach, formatted as answers to the questions in ANS

For the implementation, a skeleton Jupyter Notebook with functions that load the provided data for each task is provided. Please submit your code in the provided skeleton Jupyter Notebook. You need to upload the Jupyter Notebook (.ipynb) and a PDF version (.pdf) with the execution output. The maximum upload size is 25MB; you may need to remove some image outputs before uploading. If you are using Google Colab, you can generate the PDF via "File → Print → Save as PDF". Please generate the PDF after running all cells of your solution.

Important Notes

- All models must be trained from scratch. The use of pre-trained models is not allowed.
- LLMs can only be used for assistance regarding the code implementation. They cannot be used for answering questions. You are required to provide the model(s) and prompts used in the corresponding ANS section.